π-Former: Succinct Proofs of Transformer Inference via a SNARK-Friendly Architecture

Hideaki Takahashi
Columbia University
ht2673@columbia.edu

Abstract

Succinctly verifying transformer inference remains costly because the standard block is dominated by operations that do not have compact finite-field representations: softmax, GeLU, LayerNorm, and dense floating-point projections. Existing systems primarily optimize the proof system around this architecture, so every block continues to carry the cost of proving these non-arithmetic primitives.

We present π-Former, a transparent succinct argument for a SNARK-friendly transformer architecture. The design replaces softmax with learnable lookup-based linear attention, constrains projections to ternary matrices with per-layer scales, and expresses LayerNorm as integer constraints discharged by sumcheck and range proofs. The protocol then batches repeated block structure into model-level sumchecks, a global Lasso lookup proof, and shared range multiplicity commitments. A zerocheck fold removes the Hyrax commitments otherwise needed for LogUp inverse polynomials. We prove end-to-end soundness under the discrete-log assumption for Hyrax on BN254 G1 and implement a Rust prototype with a bit-identical PyTorch export path. The current artifact is a succinct argument rather than a zero-knowledge system; standard masking is orthogonal to the protocol presented here.

1 Introduction

Large language models are often served behind an API. A client sees an input and an output, but not the computation that produced it. Verifiable inference [1, 9, 11, 14] addresses this gap by letting the operator commit to a model and attach a succinct proof to each response. The verifier checks the proof without re-executing the model.

Transformers are a difficult target for such proofs. Softmax uses exponentials and row-wise division, GeLU is transcendental, LayerNorm uses division and square roots, and dense projections dominate the arithmetic footprint. zkLLM [14] and zkGPT [11] demonstrate that specialized lookups, constraint fusion, and parallel implementations can make standard LLM architectures substantially more practical to prove. Their results also make clear where the remaining cost comes from: the proof system is still asked to certify operations that were not designed for finite-field verification.

Our approach. π-Former takes the complementary approach of changing the model so that its operations have compact algebraic checks. Three choices drive the architecture:

- **SNARK-friendly operators.** Replace every operation without a compact arithmetic representation. Softmax becomes a kernel attention whose only non-arithmetic component is a structured table lookup; LayerNorm becomes an integer protocol provable by sumcheck plus chunked range proofs.
- **Ternary projections for free contractions.** Constrain every projection matrix to entries $\{-1, 0, +1\}$ scaled by a per-layer scalar α [10]. A degree-2 sumcheck against a ternary weight folds the equality polynomial against ternary entries with $+=$/$-=$/skip; no field multiplications appear in the contraction.
- **Learnable lookup tables.** The element-wise non-linearity ϕ is itself learnable: it is parametrised as a sum of small sub-tables and trained end-to-end. Under Lasso [13], prover cost depends only on table size and query count, not on table contents, so training adapts table values at zero proving cost.

Together these choices reduce a block to low-degree sumchecks and structured Lasso lookups.

Model-level batching. The protocol exploits the repeated structure of a depth-L transformer. Per-block claims are folded into model-level sumchecks with Fiat–Shamir random linear combinations; lookup claims are folded into global multi-instance Lasso proofs; range witnesses share one multiplicity commitment per width bucket; and residual connections are

checked by homomorphic addition of Hyrax commitments. For range proofs, a zerocheck folded into the LogUp bucket sumcheck replaces the explicit commitments to inverse polynomials, removing $B \cdot C$ Hyrax commitments per bucket.

Contributions.

1. A SNARK-friendly transformer block (§3) in which attention, normalisation, and projection each map to a sumcheck or a Lasso lookup, combining learnable lookup attention, ternary projections, and a constraint-fused integer LayerNorm.
2. A complete protocol (§4) batching repeated block structure into model-level cross-block sumchecks, global Lasso proofs, and bucketed range proofs shared across the whole model.
3. A zerocheck-folded LogUp range proof (§3.5, Theorem 5) that eliminates $B \cdot C$ Hyrax commitments per width bucket while keeping the bucket sumcheck cubic and soundness within $O(n/|\mathbb{F}|)$.
4. End-to-end soundness (§5, Theorem 8) reducing to discrete log on BN254 G1, and a Rust prototype with a bit-identical PyTorch export path (§6).

Threat model. We target non-interactive arguments in the random-oracle model. The verifier holds a verifying key vk binding the weights and lookup tables, a public input $\mathbf{x}_{\text{in}}$, and a claimed output $\mathbf{y}$, and accepts iff $\mathcal{M}_\theta(\mathbf{x}_{\text{in}}) = \mathbf{y}$ for the committed model. Soundness is computational under discrete log on BN254 G1, the standard assumption for the Hyrax PCS [17]. The prototype is not zero-knowledge: commitments are public and intermediate evaluations are revealed. We therefore claim integrity for committed inference, not privacy of all witness information. Standard masking extensions are discussed in §8.

2 Preliminaries

Notation. We work over the BN254 scalar field $\mathbb{F}$ ($r \approx 2^{254}$) and the BN254 G1 group $\mathbb{G}$. We use L for the number of blocks, T for the sequence length, d for the embedding dimension, and d_{ff} for the FFN width; the prototype is single-head ($d_h = d$). A multilinear polynomial $\widetilde{f}$ on n variables is identified with its 2^n evaluations on $\{0,1\}^n$, and the equality polynomial $\widetilde{\mathrm{eq}}(r,x) = \prod_i (r_i x_i + (1-r_i)(1-x_i))$ satisfies $\sum_x \widetilde{\mathrm{eq}}(r,x)\widetilde{f}(x) = \widetilde{f}(r)$. Real-valued tensors are quantised to integers and embedded in $\mathbb{F}$ before commitment; our training pipeline runs the same integer arithmetic, so the exported witness matches the field circuit bit-for-bit.

Transformer block. A pre-norm block [16] computes $\mathbf{X}_{\text{mid}} = \mathbf{X}_{\text{in}} + \mathrm{Att}(\mathrm{LN}(\mathbf{X}_{\text{in}}))$ and $\mathbf{X}_{\text{out}} = \mathbf{X}_{\text{mid}} + \mathrm{FFN}(\mathrm{LN}(\mathbf{X}_{\text{mid}}))$, with softmax attention $\mathrm{softmax}(\mathbf{Q}\mathbf{K}^\top/\sqrt{d})\mathbf{V}$ and LayerNorm [3] $\gamma \odot (\mathbf{x}-\mu)/\sqrt{\sigma^2+\varepsilon} + \beta$. The non-arithmetic primitives (exp, division, square root) dominate SNARK complexity in prior work.

Sumcheck. The sumcheck protocol [12,15] reduces a claim $H = \sum_{x \in \{0,1\}^n} f(x)$ for an MLE f of individual degree δ to a single evaluation $f(r)$ at random r, with soundness error $\le n\delta/|\mathbb{F}|$ and verifier cost $O(n\delta)$. π-Former uses the degree-2 variant for products of two MLEs, the cubic variant for LayerNorm variance and our zerocheck-folded range proof, and a multi-batched variant $\sum_k \lambda_k \sum_x f_k(x) g_k(x)$ for cross-block batching.

Hyrax PCS. Hyrax [17] arranges $2^n = 2^{\nu+\sigma}$ MLE evaluations as a $2^\nu \times 2^\sigma$ matrix and commits to each row via a vector Pedersen commitment over transparent generators. The scheme is computationally binding under DL on $\mathbb{G}$, requires no trusted setup, and gives $O(\sqrt{N})$ commitments, openings, and proof size. Crucially, the row-Pedersen structure is linear: $\mathsf{Com}(\mathbf{A}) + \mathsf{Com}(\mathbf{B}) = \mathsf{Com}(\mathbf{A}+\mathbf{B})$, which π-Former exploits to verify residual connections without any proof.

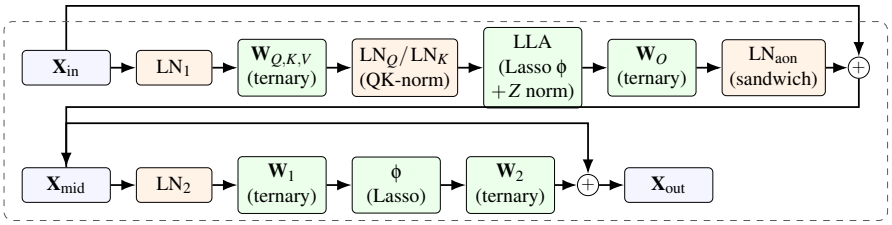
Lasso. Lasso [2,5,13] proves that N queries are consistent with a public sub-table $T \in \mathbb{F}^{2^m}$ via one batched sumcheck. A composite table that decomposes *additively* into c sub-tables (Lasso's structured condition) is committed as c small tables of size $2^{B/c}$. Lasso's defining property for our design is that prover and verifier cost depend on table size and query count, but are independent of table *contents*: training the entries of T has zero proving-cost impact.

3 The π-Former Architecture

A π-Former block (Figure 1) is a sandwich-normalised linear-attention block. With $\mathbf{X}_{\text{in}} \in \mathbb{F}^{T \times d}$, the block computes

$$
\begin{aligned}
\mathbf{X}_{\text{n1}} &= \mathrm{LN}_1(\mathbf{X}_{\text{in}}),\\
\mathbf{Q},\mathbf{K},\mathbf{V} &= \mathbf{X}_{\text{n1}}\mathbf{W}_{Q,K,V} \ \ \text{(ternary)},\\
\mathbf{Q}^{\text{n}},\mathbf{K}^{\text{n}} &= \mathrm{LN}_Q(\mathbf{Q}),\ \mathrm{LN}_K(\mathbf{K}),\\
\mathbf{N} &= \phi(\mathbf{Q}^{\text{n}})\big(\phi(\mathbf{K}^{\text{n}})^\top \mathbf{V}\big),\\
Z_t &= \phi(\mathbf{q}^{\text{n}}_t)^\top \textstyle\sum_s \phi(\mathbf{k}^{\text{n}}_s),\\
\mathbf{Y}^{\text{att}}_{tj} &= \lfloor S \cdot \mathbf{N}_{tj}/Z_t \rfloor,\\
\mathbf{X}_{\text{mid}} &= \mathbf{X}_{\text{in}} + \mathrm{LN}_{\text{aon}}(\mathbf{Y}^{\text{att}}\mathbf{W}_O + \mathbf{b}_O),\\
\mathbf{X}_{\text{out}} &= \mathbf{X}_{\text{mid}} + \phi(\mathrm{LN}_2(\mathbf{X}_{\text{mid}})\mathbf{W}_1)\mathbf{W}_2.
\end{aligned}
$$

After L such blocks the model applies a final LayerNorm and a ternary LM head; the model contains $5L+1$ LayerNorms in total. The scale S re-introduces fractional precision before the integer floor division.



Proof structure. LayerNorms, ternary projections, attention normalisation, Lasso lookups, and range checks are each batched at model level. Residual additions are verified by homomorphic Hyrax addition and require no separate sub-proof.

Figure 1: A π-Former block. Green nodes are algebraic sub-protocols; orange nodes are LayerNorms. QK-norm bounds lookup inputs before ϕ, sandwich norm bounds the attention output before the residual, and all linear maps use ternary weights.

Two of the five LayerNorms per block are not in a textbook transformer but are essential for our integer pipeline. *QK-norm* (LN_Q, LN_K) pins the range of $\mathbf{Q}, \mathbf{K}$ so that $\phi(\mathbf{Q}^{\text{n}}), \phi(\mathbf{K}^{\text{n}})$ and Z stay in the lookup-table range under any per-layer ternary scale. *Sandwich norm* (LN_{aon}) absorbs the unbounded magnitude of linear attention before the residual: softmax is row-stochastic and naturally bounded, but $\phi(\mathbf{Q})(\phi(\mathbf{K})^\top \mathbf{V})$ is not, and an unnormalised residual destabilises downstream LayerNorms. Both share one LN sub-protocol (§3.4); their only cost is extra LN witnesses, folded into the model-level batched LN proof and the bucketed range batch.

3.1 Learnable Lookup Attention

Softmax attention requires T^2 exponentials and T row-wise divisions. We replace the kernel with a positive-definite inner product $\kappa(\mathbf{q}, \mathbf{k}) = \langle \phi(\mathbf{q}), \phi(\mathbf{k}) \rangle$ on the post-QK-norm tensors, where $\phi : \mathbb{R}^{d_h} \to \mathbb{R}^{d_h}$ is an element-wise structured lookup (§3.2). The block computes the numerator $\mathbf{N}_{tj} = \phi(\mathbf{q}_t^{\text{n}})^\top (\sum_s \phi(\mathbf{k}_s^{\text{n}}) \mathbf{v}_s^\top)_j$ and the per-row denominator $Z_t = \phi(\mathbf{q}_t^{\text{n}})^\top \sum_s \phi(\mathbf{k}_s^{\text{n}})$. The context matrix $\mathbf{M} = \sum_s \phi(\mathbf{k}_s^{\text{n}}) \mathbf{v}_s^\top \in \mathbb{F}^{d_h \times d_h}$ is computed once per block, giving $O(T d_h^2)$ prover cost instead of $O(T^2 d_h)$ [7].

Floor-division proof for row normalisation. π-Former proves $\mathbf{Y}^{\text{att}}_{tj} = \lfloor S\mathbf{N}_{tj} / Z_t \rfloor$ in-circuit. The witness includes auxiliary tensors rem and diff with $\text{rem}_{tj} = S\mathbf{N}_{tj} - Z_t \mathbf{Y}^{\text{att}}_{tj}$ and $\text{diff}_{tj} = Z_t - 1 - \text{rem}_{tj}$, all committed via Hyrax. At a random $r_{\text{norm}} \in \mathbb{F}^{t_{\text{bits}} + d_{\text{bits}}}$ one cubic multi-batched sumcheck merges $S\mathbf{N} - \text{rem} - Z\mathbf{Y}^{\text{att}} = 0$ and $\lambda(Z - 1 - \text{rem} - \text{diff}) = 0$ under a Fiat–Shamir scalar λ. Range proofs on rem, diff (in the 64-bit bucket of §3.5) enforce both to be non-negative, which combined with the second identity gives $0 \le \text{rem} < Z$ — the unique floor-division witness. A second sumcheck binds Z_t to the committed $\phi(\mathbf{Q}^{\text{n}}), \phi(\mathbf{K}^{\text{n}})$; a causal variant binds Z to a prefix sum.

Binding lookups to post-norm tensors. The Lasso queries for ϕ are evaluated on $\mathbf{Q}^{\text{n}}, \mathbf{K}^{\text{n}}$, not the pre-norm $\mathbf{Q}, \mathbf{K}$. The prover commits both $C_{\mathbf{Q}}, C_{\mathbf{K}}$ (used by the QKV sumcheck) and $C_{\mathbf{Q}^{\text{n}}}, C_{\mathbf{K}^{\text{n}}}$ (used as ϕ inputs); the global ϕ Lasso binds its query indices to the post-norm commitments via one shared Hyrax batch open, and the LN sub-proof for LN_Q, LN_K ties pre- and post-norm together.

3.2 Additive sub-table feature map

Rather than fix a kernel, π-Former parametrises ϕ as a sum of learnable sub-table lookups. With input bit-width B, decomposition depth $c \mid B$, and chunk width $m = B/c$, define the integer index $x_{\text{int}} = \text{clamp}(\lfloor x/s \rfloor, 0, 2^B - 1)$ and chunk extractor $\chi_i(x_{\text{int}}) = (x_{\text{int}} \gg im) \,\&\, (2^m - 1)$. Then

$$\phi(x) \;=\; \sum_{i=0}^{c-1} T_i\big[\chi_i(x_{\text{int}})\big], \tag{1}$$

where $T_0, \ldots, T_{c-1} \in \mathbb{R}^{2^m}$ are learnable sub-tables. They are initialised so $\sum_i T_i \approx \text{GELU}$ and trained end-to-end via the straight-through estimator [4]. For $B = 16, c = 2$ this replaces a 65,536-entry monolithic table with two 256-entry sub-tables, matching Lasso's structured-decomposability condition (§2). Because Lasso prover cost is independent of table values, training the entries of every T_i end-to-end imposes zero proving-cost penalty.

3.3 Ternary projections

Following BitNet b1.58 [10], every projection matrix ($\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O, \mathbf{W}_1, \mathbf{W}_2$, and the LM head) is constrained to entries in $\mathcal{T} = \{-1, 0, +1\}$ scaled by a learnable per-layer scalar $\alpha \in \mathbb{F}$, so the forward pass is $\mathbf{y} = \alpha(\mathbf{x}\mathbf{W}) + \mathbf{b}$ with $\mathbf{W} \in \mathcal{T}^{m \times n}$. Training uses the magnitude-thresholded $w \mapsto \text{sign}(w) \cdot [|w| > 0.7\overline{|w|}]$ map with the straight-through estimator; α is multiplied in after the matrix product so the in-circuit weight stays ternary.

In the SNARK, the contraction $\mathbf{Y}(r_t, r_d) - \mathbf{b}(r_d) = \alpha \sum_k \mathbf{X}(r_t, k)\mathbf{W}(k, r_d)$ is a degree-2 sumcheck that materialises only the cheap vector $\alpha \cdot \mathbf{W}(\cdot, r_d)$. The prover folds the equality polynomial against ternary entries with $+=$/$-=$/skip; no field multiplication appears in the contraction.

3.4 Constraint-fused LayerNorm

Let $\mathbf{x} \in \mathbb{Z}^d$ be a row, $\text{sum\_x} = \sum_j x_j$, $\text{sq\_sum\_x} = \sum_j x_j^2$, and $\sigma = \lfloor \sqrt{\text{var\_x}}/d \rfloor$. The integer variance is $\text{var\_x} = d(d\,\text{sq\_sum\_x} - \text{sum\_x}^2)$, which expands $\sum_j (dx_j - \text{sum\_x})^2$ without taking sum_x^2 outside the sumcheck. π-Former proves a LayerNorm without divisions or square roots in five sub-protocols:

1. **Mean** (degree-2 sumcheck for $\widetilde{\text{sum\_x}}$).
2. **Square-sum** (cubic sumcheck for $\widetilde{\text{sq\_sum\_x}}(r_t) = \sum_j x_{\text{col}}(j)^3$ with three copies of x_{col}, keeping the round polynomial cubic).
3. **Sigma residual** (cubic multi-batched sumcheck combining $\sum_i \widetilde{\text{eq}}\widetilde{\text{sum\_x}}(i)^2$ and $\sum_i \widetilde{\text{eq}}(d\widetilde{\sigma}(i))^2$ under a Fiat–Shamir scalar λ_{sig}; the verifier reconstructs var_x algebraically).
4. **Floor-sqrt** (range-check $\text{var\_x} - (d\sigma)^2 \geq 0$ and $(d\sigma + d)^2 - 1 - \text{var\_x} \geq 0$ via §3.5).
5. **Y fusion** (at one random (r_{y_t}, r_{y_d}), check $\gamma \odot (d\mathbf{x} - \text{sum\_x}) + \beta \cdot d\sigma \approx d\sigma \cdot \mathbf{y}$ via a range proof on the lo/hi residuals; the $\gamma\mathbf{X}$ and $\sigma\mathbf{Y}$ legs share one multi-batched cubic sumcheck).

Per LN the prover commits three internal Hyrax values $(\text{sum\_x}, \text{sq\_sum\_x}, \sigma)$; the squared quantities are bound algebraically by step 3 and need no separate commitment. The verifier evaluates γ, β from the VK in $O(d)$ per layer.

Model-level LN batching. After Phase 1 has absorbed all $5L+1$ LN IO commitments, one ProveLNsBatched call folds mean / variance / σ-floor / Y-fusion sub-claims of every LN into a single batched cubic sumcheck under Fiat–Shamir powers of an independent challenge. Soundness follows from the same Schwartz–Zippel argument as cross-block batching (Theorem 3) with L replaced by $5L+1$.

3.5 Globally batched range proof

The range proof shows $V[i] \in [0, 2^{\text{bits}})$ via chunked LogUp [6] over an identity table, with chunk width 16 and bucket widths $\{32, 64\}$. LayerNorm σ/y witnesses use the 64-bit bucket; each non-empty bucket produces one multiplicity commitment m_{com}.

For B witnesses in a bucket with $C = \lceil \text{bits}/16 \rceil$ chunks each, Phase 1 commits all $V_0^{(b)}, \ldots, V_{C-1}^{(b)}$ via Hyrax and merges chunk arrays into one global multiplicity vector $m[v] = \sum_{b,c,i} [\text{chunk}^{(b,c)}[i] = v]$ committed once. Phase 2 runs one degree-2 sumcheck per witness binding $V^{(b)}$ to a random $r_v^{(b)}$ and verifies $V^{(b)}(r_v^{(b)}) = \sum_c 2^{16c} V_c^{(b)}(r_v^{(b)})$ through one Hyrax batch open. Phase 3 opens $m(r_m)$ once and checks the LogUp identity against $T_{\text{id}}[i] = i$. The transcript-ordering invariant (Theorem 5) is that m_{com} is absorbed before any Phase 2 sumcheck challenge.

Zerocheck-folded inverse polynomial. The LogUp identity requires per-chunk inverse polynomials $h_c[i] = 1/(\alpha - V_c[i])$ for a Fiat–Shamir challenge α. A naïve implementation Hyrax-commits each h_c and opens it alongside the chunk commitments, costing $B \cdot C$ extra Pedersen MSMs per bucket. We avoid those commitments by folding a zerocheck of

$$h(x) \cdot (\alpha - V(x)) - 1 \equiv 0 \qquad (x \in \{0,1\}^n)$$

into the bucket's existing cubic sumcheck. Concretely, after the bucket claim $\Lambda = \sum_p w_p \cdot \big(\sum_i h_p[i] + \beta \cdot n_{\text{pad}}\big)$ is absorbed, the verifier samples $\gamma_{\text{fold}} \in \mathbb{F}$ and $r_{zc} \in \mathbb{F}^n$, and the prover proves

$$\sum_x \Big[w(x)h(x)\big(1 + \beta(\alpha - V(x))\big) + \gamma_{\text{fold}} \widetilde{\text{eq}}(r_{zc}, x)\big(h(x)(\alpha - V(x)) - 1\big) \Big] = \Lambda.$$

The first term is the standard LogUp bucket claim; the second is the zerocheck term, which sums to zero on the boolean hypercube iff $h = 1/(\alpha - V)$ pointwise. The integrand remains cubic in (h, V, w, eq), so per-round prover cost is unchanged. We pre-fill h's pair-padding slots with α^{-1} so that the zerocheck term vanishes there ($\alpha^{-1} \cdot \alpha - 1 = 0$), keeping the bucket claim well-formed without an indicator mask.

Crucially, Λ is absorbed into the transcript *before* γ_{fold} and r_{zc} are sampled. The terminal $h(r_{\text{full}})$ is sent as a scalar (not opened against any commitment) and is bound only by the sumcheck's round-polynomial consistency together with the combined-identity check; the chunk evaluation $V(r_{\text{full}})$ remains bound to the Hyrax-committed V_c via the per-chunk openings at the bucket challenge r_k.

The model-level prover routes *all* range witnesses across the model into one bucketed list: $10L+2$ LN witnesses (σ, y for the five LNs per block, plus the final LN's two), and $2L$ further witnesses (rem, diff per block) when normalised attention is enabled. Sharing m_{com} per bucket saves $(B-1)\sqrt{2^{16}}$ G1 MSMs per bucket; the zerocheck fold removes a further $B \cdot C$ Hyrax commits and as many opens, with savings scaling with L rather than capped at the per-block level.

4 The π-Former Protocol

This section describes the setup, prover, and verifier at the level of the cross-block protocol structure. Detailed per-block and per-LN pseudocode is in Appendix A.

4.1 Setup

Algorithm 1 π-Former. $\mathsf{Setup}(\theta)$

Input: ternary weights and biases θ, lookup sub-tables $T_0, \ldots, T_{c-1}$, dimensions.
Output: $(\mathsf{pk}, \mathsf{vk})$.

1: derive transparent Hyrax generators g_j from SHA3 of public seeds
2: $C_{\mathbf{W}} \leftarrow \mathsf{Com}(\mathbf{W})$ for each weight matrix; $C_{T_i} \leftarrow \mathsf{Com}(T_i)$ for each sub-table
3: $\mathsf{vk} \leftarrow \{C_{\mathbf{W}}\} \cup \{C_{T_i}\} \cup \{\gamma, \beta\} \cup \{g_j\}$
4: $\mathsf{pk} \leftarrow \mathsf{vk} \cup \{\text{raw ternary arrays and sub-tables}\}$

The verifying key contains no raw weights, only commitments. The proving key additionally carries the ternary arrays so the prover can fold the equality polynomial against them in $O(Td)$ without field multiplications.

4.2 Model-level prover

Algorithm 2 π-Former. $\mathsf{Prove}(\mathsf{pk}, W, \mathsf{inst})$ outline

1: *Phase 1.* For each block ℓ: commit intermediates (per-block LN IO, ϕ inputs, attention auxiliaries); absorb in fixed order; chain residuals by Hyrax addition.
2: *Phase 2.* Draw one global random point $r_{td} \leftarrow \mathsf{FS.challenge}$ shared by all cross-block sumchecks.
3: *Phase 3.* Run four cross-block sumchecks (QKV, output projection, attention out, attention context) via CrossBatch.
4: *Phase 4.* (When normalised attention is enabled) prove $S\mathbf{N} - \mathrm{rem} - Z\mathbf{Y}^{\mathrm{att}} = 0$ and $Z - 1 - \mathrm{rem} - \mathrm{diff} = 0$ in one cubic multi-batched sumcheck; bind Z to $\phi(\mathbf{Q}^{\mathrm{n}}), \phi(\mathbf{K}^{\mathrm{n}})$.
5: *Phase 5.* Commit FFN activations; prove the global $\phi(\mathbf{M})$ Lasso (committed-output) and two FFN cross-block sumchecks.
6: *Phase 6.* Run ProveRangeBatched once per width bucket over all witnesses, and ProveLNsBatched once over all $5L+1$ LNs.
7: *Phase 7.* Prove the LM head; drain deferred-accumulator batching challenges and finalise in one MSM pair per Hyrax-parameter group; emit grouped batched opens at r_{td}, the per-protocol terminal points, and (when normalised) r_{norm}.
8: *Phase 8.* Prove the global $\phi(\mathbf{Q}^{\mathrm{n}}), \phi(\mathbf{K}^{\mathrm{n}})$ Lasso, binding indices to the committed post-norm tensors.

Three structural choices drive the design. *(i)* A single global random point r_{td} is drawn after all per-block commitments are absorbed, so the six cross-block sumchecks share it. *(ii)* All final Hyrax openings flow through deferred batch accumulators (one per Hyrax-parameter group), each finalised in one MSM pair via random-linear-combination batching. *(iii)* Residual connections require no proof: by linearity of Hyrax (Theorem 4), $C_{\mathbf{X}_{\mathrm{mid}}} = C_{\mathbf{X}_{\mathrm{in}}} + C_{\mathrm{LN}_{\mathrm{aon}}.y}$ is computed by the verifier from the existing commitments.

Cross-block batched sumcheck. The primitive that powers Phases 3–5 reduces L per-block claims $\{H_\ell\}$ over per-block MLEs $\{f_\ell, g_\ell\}$ on $\mathbb{F}^n$ to a single sumcheck on $P(x) = \sum_\ell \eta^{\ell-1} f_\ell(x) g_\ell(x)$ at a Fiat–Shamir challenge η drawn after every per-block constant is absorbed.

Algorithm 3 CrossBatch$(\{f_\ell, g_\ell, H_\ell\}_\ell, \mathsf{FS})$

1: absorb $\{H_\ell\}$ and per-block constants; $\eta \leftarrow \mathsf{FS.challenge}$
2: $H^\star \leftarrow \sum_\ell \eta^{\ell-1} H_\ell$; run standard sumcheck on P to a final point $r \in \mathbb{F}^n$
3: verifier checks $P(r) = \sum_\ell \eta^{\ell-1} f_\ell(r) g_\ell(r)$ against per-block evaluations opened later in a batched open

By Theorem 3 this replaces L independent sumcheck transcripts with one of length n at an additive $(L-1)/|\mathbb{F}|$ soundness cost.

Quantisation binding. Lasso queries are integer indices $\mathsf{idx}_j \in [0, 2^B)$, while committed input tensors carry signed field elements. Two quantisation sub-proofs bridge this gap. With public scales $(s_{\mathrm{num}}, s_{\mathrm{den}})$ and zero-point $\mathsf{zp} = 2^{B-1}$, the prover commits the remainder $\mathsf{rem}_j = r_j s_{\mathrm{den}} + \lfloor s_{\mathrm{num}}/2 \rfloor - s_{\mathrm{num}}(\mathsf{idx}_j - \mathsf{zp})$, adds rem to the global range batch with bit-width $\log_2 s_{\mathrm{num}}$, and at a Fiat–Shamir point $\mathbf{r}$ opens $\widetilde{r}$ and $\widetilde{\mathsf{rem}}$ via batched Hyrax. The verifier evaluates the public index MLE $\widetilde{\mathsf{idx}}(\mathbf{r})$ directly and checks

$$\widetilde{r}(\mathbf{r})\, s_{\mathrm{den}} + \tfrac{s_{\mathrm{num}}}{2} \widetilde{1}(\mathbf{r}) \stackrel{?}{=} s_{\mathrm{num}} \big(\widetilde{\mathsf{idx}}(\mathbf{r}) - \mathsf{zp}\, \widetilde{1}(\mathbf{r}) \big) + \widetilde{\mathsf{rem}}(\mathbf{r}). \quad (2)$$

Combined with the remainder range proof, this pins every public idx_j to the unique integer consistent with the committed raw value r_j.

Lasso instances. Both model-level Lasso instances are *committed-output* Lassos: the prover absorbs every Hyrax commitment to a lookup output before batching challenges are drawn, and an inner sumcheck binds the combined grand sum to the committed outputs at the sumcheck terminal. Lookup indices are bound to the committed post-norm input tensors via one shared Hyrax batch open, so a cheating prover can neither pick query indices independently of the inputs nor mix pre- and post-norm tensors.

4.3 Verifier

Algorithm 4 π-Former. $\mathsf{Verify}(\mathsf{vk}, \pi, \mathsf{inst}, \mathbf{x}_{\text{in}}, \mathbf{y})$ outline

1: re-commit $\mathbf{x}_{\text{in}}$ and check against $\pi.C_{\mathbf{x}_{\text{in}}}$; initialise deferred Hyrax accumulators
2: mirror Phase 1 transcript absorption, computing residual commitments by Hyrax addition
3: draw r_{td}; verify the four cross-block sumchecks
4: (when normalised) verify the attention-normalisation sumcheck and the auxiliary Z sumcheck
5: verify the global FFN Lasso and the two FFN cross-block sumchecks
6: verify the bucketed range batch (one m_{com} per width)
7: verify the model-level batched LN proof over all $5L+1$ LNs
8: draw r_{lm}, evaluate $\widetilde{\mathbf{y}}(r_{\text{lm}})$ from the public output, verify the LM-head sumcheck; chain its input-side claim into the td accumulator against $\pi.C_{\text{final\_ln\_out}}$
9: drain deferred-accumulator batching challenges and finalise in one MSM pair per Hyrax-params group; verify all grouped batched openings
10: verify the global $\phi(\mathbf{Q}^{\text{n}}), \phi(\mathbf{K}^{\text{n}})$ Lasso; return ACCEPT

5 Soundness

We state the main soundness theorems and outline the arguments here; full proofs appear in Appendix B.

Lemma 1 (Sumcheck soundness)**.** *A sumcheck for an MLE of individual degree $\leq \delta$ over $\{0,1\}^n$ has soundness error $\leq n\delta/|\mathbb{F}|$.*

Lemma 2 (Hyrax binding)**.** *Under discrete log on $\mathbb{G}$, the Hyrax PCS is computationally binding.*

Theorem 3 (Cross-block batching is sound)**.** CROSSBATCH *(Alg. 3) reduces L per-block claims to one batched claim $\sum_\ell \eta^{\ell-1} H_\ell$ with total soundness error $\leq (L-1)/|\mathbb{F}| + n\delta/|\mathbb{F}|$.*

Sketch. η is drawn after every per-block claim is absorbed, so the prover is committed to $\{H_\ell\}$ before seeing η. If any claim is false, the difference polynomial in η has degree $\leq L-1$ and is caught by Schwartz–Zippel with probability $\geq 1-(L-1)/|\mathbb{F}|$; the inner sumcheck adds $n\delta/|\mathbb{F}|$. □

Theorem 4 (Homomorphic residuals)**.** *For Hyrax commitments $C_{\mathbf{A}}, C_{\mathbf{B}}$, the verifier-computed commitment $C_{\mathbf{A}} + C_{\mathbf{B}}$ binds $\mathbf{A}+\mathbf{B}$.*

Sketch. By bilinearity of MSM over public generators, $C_{\mathbf{A}} + C_{\mathbf{B}} = \mathsf{Com}(\mathbf{A}+\mathbf{B})$; binding follows from Lemma 2. □

Theorem 5 (Globally batched range proof is sound)**.** *Sharing one m_{com} per width bucket across all range witnesses prevents any chunk value outside $[0, 2^{16})$. The zerocheck-folded inverse polynomial binds h to $1/(\alpha - V)$ on the boolean hypercube* without *committing h, with soundness error $O(n/|\mathbb{F}|)$ per bucket where n is the bucket variable count.*

Sketch. Two parts. *Chunk-value binding:* m_{com} is absorbed before any sumcheck challenge, so any chunk $v^* \geq 2^{16}$ has no matching T_{id} entry, breaking the LogUp identity. *Inverse-polynomial binding:* for malicious $\tilde{h} \neq 1/(\alpha - V)$, the zerocheck residual $\tilde{s}(x) = \tilde{h}(x)(\alpha - V(x)) - 1$ is non-zero on the hypercube, hence non-zero as a polynomial of degree $\leq 2n$; Schwartz–Zippel gives $\Pr_{r_{zc}}[\tilde{s}(r_{zc}) = 0] \leq 2n/|\mathbb{F}|$, the linear-in-$\gamma_{\text{fold}}$ check adds $1/|\mathbb{F}|$, and sumcheck adds $3n/|\mathbb{F}|$. Pair-padding $\tilde{h} = \alpha^{-1}$ keeps padding contributions zero in both terms. Full proof in Appendix B. □

Theorem 6 (Model-level batched LayerNorm is sound)**.** *The single* PROVELNSBATCHED *call covering all $5L+1$ LNs is as sound as $5L+1$ independent LN sub-proofs.*

Sketch. Apply Theorem 3 with L replaced by $5L+1$. □

Theorem 7 (Attention-normalisation sumcheck is sound)**.** *The cubic multi-batched sumcheck for $S\mathbf{N} - \text{rem} - Z\mathbf{Y}^{\text{att}} = 0$ and $\lambda(Z - 1 - \text{rem} - \text{diff}) = 0$ combined with range proofs on* rem, diff *and the auxiliary Z sumcheck soundly enforces $\mathbf{Y}^{\text{att}}_{tj} = \lfloor S\mathbf{N}_{tj}/Z_t \rfloor$ pointwise.*

Sketch. Range proofs give $0 \leq \text{rem} < Z$ via the second identity; the first identity then expresses $Z\mathbf{Y}^{\text{att}}$ as the unique floor-division quotient. Both checks at one random point cost cubic sumcheck error plus $O(1)/|\mathbb{F}|$ batching error; the auxiliary Z sumcheck binds Z_t to the committed $\phi(\mathbf{Q}^{\text{n}}), \phi(\mathbf{K}^{\text{n}})$. □

Theorem 8 (π-Former end-to-end soundness)**.** *For any PPT adversary $\mathcal{A}$,*

$$\Pr[\mathsf{Verify} = \mathsf{ACCEPT} \wedge \mathcal{M}_\theta(\mathbf{x}_{\text{in}}) \neq \mathbf{y}] \leq \frac{m_{\text{sc}} n_{\max} \delta_{\max} + m_{\text{batch}}(L-1) + m_{\text{open}}}{|\mathbb{F}|} + \varepsilon_{\text{DL}}.$$

where m_{sc} is the post-batching sumcheck count, $m_{\text{batch}} = 6$, m_{open} covers RLC checks, $n_{\max} \leq 64$, $\delta_{\max} \leq 3$, and ε_{DL} is $\mathcal{A}$'s DL advantage on $\mathbb{G}$. For typical parameters the statistical term is below 2^{-240}.

Sketch. Hybrid argument over the L blocks: any forged Hyrax open yields a DL solution (Lemma 2); residual chaining is sound by Theorem 4; per-block algebra is sound by Theorems 5, 6, 7, and 3; the committed-output Lasso is sound by Lemmas 2–1. A union bound over all sub-protocols yields the stated error. □

5.1 Privacy

π-Former currently provides integrity for committed inference, not zero knowledge. The verifier sees public commitments and the sumcheck-opening values needed for verification. Hiding all witness information would require blinding

Stage	Time (ms)
Commit + LN witness extraction	23.0
Range proof (LN bucket, 32-bit)	389.2
Range proofs (8-bit buckets)	240.0
Cross-block linear/attention checks	65.0
Global ϕ Lasso	110.0
Model-level batched LN proof	71.0
LM head + accumulator finalisation	14.3
Total prover time	**912.5**

Table 1: Prover-side breakdown for the 2-layer prototype. The zerocheck-folded range proof reduces the 32-bit LayerNorm bucket from 435.9 ms to 389.2 ms (-10.7%) by eliminating $B{\cdot}C$ Hyrax commitments per bucket.

committed polynomials and using zero-knowledge variants of sumcheck and Hyrax. These changes are orthogonal to the algebraic reductions in this paper, but are not implemented in the present prototype.

6 Implementation

We implement π-Former in approximately 19,000 lines of Rust over BN254 (`ark-bn254`), with a matched PyTorch training pipeline. The pipeline implements ternary projections, the structured-lookup activation, and linear attention with the same integer arithmetic used by the Rust circuit, so a witness exported from PyTorch is accepted verbatim by the Rust prover. The reference implementation is single-head and supports optional end-to-end causal masking via three cubic-multi-batched sub-protocols whose third multiplicand absorbs the causal indicator polynomial; multi-head and zero-knowledge masking are out of scope.

7 Prototype Evaluation

Setup. We evaluate the Rust prototype on a small end-to-end model: $L=2$, $T=16$, $d=64$, $d_{ff}=256$, vocabulary 256, and 8-bit activations. Timings use the release build on one x86-64 CPU thread, with no SIMD, GPU MSM, or distributed sumcheck. The goal of this section is to validate the protocol decomposition and the range proof ablation, not to claim competitiveness with optimized GPU systems such as zkLLM [14] or zkGPT [11].

Zerocheck fold ablation. Replacing the explicit Hyrax commitment to each LogUp inverse polynomial by the zerocheck fold of §3.5 eliminates $B{\cdot}C$ Hyrax MSMs per bucket. On the prototype this translates to a 46.7 ms reduction in the 32-bit LN bucket (-10.7% on that phase, Table 1). The savings scale with the number of range witnesses; for larger models the absolute reduction grows roughly linearly in L.

Verifier. Verifier wall-clock on the same prototype is dominated by Hyrax opening checks and accumulator finalisation; algebraic sumcheck checks are small by comparison. This matches the protocol design: once per-block claims are folded, verification cost is governed primarily by the number and size of polynomial openings rather than by model depth.

8 Discussion and Limitations

Scope. The prover proves the full normalised LLA output $\mathbf{Y}^{\text{att}} = \lfloor S\phi(\mathbf{Q}^{\text{n}})(\phi(\mathbf{K}^{\text{n}})^{\top}\mathbf{V})/Z \rfloor$, the ternary output projection, and the sandwich-norm LN_{aon} before the residual. Causal masking is fully supported. The main limitations are single-head attention, no zero-knowledge masking, and no deployment-scale quality evaluation. Because π-Former changes the architecture by replacing softmax with a learnable kernel and constraining weights to ternary values, model quality must be evaluated against matched baselines under the same parameter budget, dataset, and quantisation regime.

Engineering path. The reference prover is CPU-only and single-threaded. The dominant kernels–cross-block sumcheck, multi-instance Lasso, and batched Hyrax opening–are data-parallel and should benefit from the same GPU MSM and parallel-sumcheck engineering used in recent LLM proof systems [11, 14]. Streaming generation is a natural setting for folding: the running context matrix $\phi(\mathbf{K})^{\top}\mathbf{V}$ can serve as the accumulator, with one proof step per generated token.

9 Related Work

Verifiable ML. zkCNN [9] gave the first GKR-based proof system for neural-network inference, focused on convolutional networks; [1] extends ZK to proofs of training. zkLLM [14] adapts a custom IOP to softmax LLM inference with `tlookup`-style structured lookups and a tailored `zkAttn` attention sub-protocol; zkGPT [11] pairs Plonky2 with constraint fusion across the operations of a transformer block. Both target the standard softmax + GeLU + LayerNorm stack and pay a fixed cost on every non-arithmetic operation. π-Former departs by co-designing the model and proof system: every operation maps to a sumcheck or a Lasso lookup by construction, and the model recovers expressivity through learnable lookup tables that are free under Lasso.

Sumcheck and lookup arguments. Sumcheck underlies modern transparent SNARKs [12, 15]. Lasso provides a practical lookup argument with sub-linear prover cost on structured-decomposable tables [2, 5, 13]; LogUp [6] backs the globally batched range proof; Hyrax [17] is a transparent discrete-log polynomial commitment with $O(\sqrt{N})$ proofs.

Linear attention and ternary networks. Linear attention [7,8,18] replaces softmax by a feature-map kernel. BitNet b1.58 [10] popularised ternary projections for inference energy; π-Former re-purposes the same primitive for SNARK proving cost.

10 Conclusion

π-Former demonstrates that co-designing the model with the proof system yields a transformer block whose every operation is either a sumcheck or a small Lasso lookup. Softmax becomes a kernel attention with a learnable additive-lookup feature map; projections become ternary, eliminating field multiplications from weight contractions; LayerNorm becomes an integer protocol provable through sumcheck and chunked range proofs. On top of this architecture we build a transparent BN254 SNARK that batches all per-block subprotocols across L blocks and replaces the LogUp inverse-polynomial commitment by a zerocheck fold, with end-to-end soundness reducing to discrete log on BN254 G1. We leave a deployment-quality empirical comparison and the remaining engineering steps (multi-head, zero-knowledge masking, GPU MSM, on-chain verifier) to future work.

References

[1] Kasra Abbaszadeh, Christodoulos Pappas, Jonathan Katz, and Dimitrios Papadopoulos. Zero-knowledge proofs of training for deep neural networks. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 4316–4330, 2024.

[2] Arasu Arun, Srinath Setty, and Justin Thaler. Jolt: Snarks for virtual machines via lookups. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 3–33. Springer, 2024.

[3] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[4] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

[5] Oleg Fomenko and Anton Levochko. Understanding lasso: A novel lookup argument protocol. *Cryptology ePrint Archive*, 2025.

[6] Ulrich Haböck. Improving logarithmic derivative lookups using gkr. *Cryptology ePrint Archive*, 2022.

[7] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pages 5156–5165. PMLR, 2020.

[8] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*, 2020.

[9] Tianyi Liu, Xiang Xie, and Yupeng Zhang. Zkcnn: Zero knowledge proofs for convolutional neural network predictions and accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 2968–2985, 2021.

[10] Shuming Ma, Hongyu Wang, Shaohan Huang, Xingxing Zhang, Ying Hu, Ting Song, Yan Xia, and Furu Wei. Bitnet b1. 58 2b4t technical report. *arXiv preprint arXiv:2504.12285*, 2025.

[11] Wenjie Qu, Yijun Sun, Xuanming Liu, Tao Lu, Yanpei Guo, Kai Chen, and Jiaheng Zhang. {zkGPT}: An efficient non-interactive zero-knowledge proof framework for {LLM} inference. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 2045–2063, 2025.

[12] Srinath Setty. Spartan: Efficient and general-purpose zksnarks without trusted setup. In *Annual International Cryptology Conference*, pages 704–737. Springer, 2020.

[13] Srinath Setty, Justin Thaler, and Riad Wahby. Unlocking the lookup singularity with lasso. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 180–209. Springer, 2024.

[14] Haochen Sun, Jason Li, and Hongyang Zhang. zkllm: Zero knowledge proofs for large language models. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 4405–4419, 2024.

[15] Justin Thaler. *Proofs, arguments, and zero-knowledge*, volume 4. Foundations and Trends in Privacy and Security, 2022.

[16] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[17] Riad S Wahby, Ioanna Tzialla, Abhi Shelat, Justin Thaler, and Michael Walfish. Doubly-efficient zksnarks without trusted setup. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 926–943. IEEE, 2018.

[18] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.

A Detailed algorithms

A.1 Per-block Phase 1

Algorithm 5 PHASE1$(W_\ell, C_{\text{cur}}, \mathsf{pk}_\ell, \mathsf{FS})$

Input: block witness, current input commitment, block PK.
Output: intermediate commitments; no per-block LN sumcheck or range proof.

commit pre-/post-QK-norm $\mathbf{Q}, \mathbf{K}, \mathbf{V}$: $C_{\mathbf{Q}}, C_{\mathbf{K}}, C_{\mathbf{V}}, C_{\mathbf{Q}^{\text{n}}}, C_{\mathbf{K}^{\text{n}}}$
commit LN outputs and FFN/O-proj outputs; (when normalised) commit $C_{\mathbf{N}}, C_{\mathbf{Y}^{\text{att}}}, C_Z, C_{\text{rem}}, C_{\text{diff}}$
absorb LN_1 IO; absorb $C_{\mathbf{Q}}, C_{\mathbf{K}}, C_{\mathbf{V}}$ + (optional) attn-norm aux commits
absorb LN_Q/LN_K IO; $C_{\mathbf{O}_{\text{att}}}$; LN_{aon} IO
$C_{\mathbf{X}^\ell_{\text{mid}}} \leftarrow C_{\text{cur}} + C_{\text{LN}_{\text{aon}}.y}$
absorb LN_2 IO; $C_{\mathbf{O}_{\text{ffn}}}$
return all commitments

A.2 Constraint-fused LayerNorm prover (one LN witness)

Algorithm 6 PROVELN$(W, \{C_{\mathbf{x}}, C_{\mathbf{y}}\}, (\mathsf{rps}_\sigma, \mathsf{rps}_y), \mathsf{FS})$

Input: witness $\{\mathbf{x}, \mathbf{y}, \text{sum\_x}, \text{sq\_sum\_x}, \sigma\}$ and pre-issued range proofs $\mathsf{rps}_\sigma, \mathsf{rps}_y$ from the global range batch.

commit $C_{\text{sum\_x}}, C_{\text{sq\_sum\_x}}, C_\sigma$; absorb IO and internals
$r_t \leftarrow \mathsf{FS.challenge}(t_{\text{bits}})$
degree-2 sumcheck for $\widetilde{\text{sum\_x}}(r_t) = \sum_j x_{\text{col}}(j)$
cubic sumcheck for $\widetilde{\text{sq\_sum\_x}}(r_t) = \sum_j x_{\text{col}}(j)^3$ via three copies of x_{col}
cubic multi-batched sumcheck combining $\widetilde{\text{sum\_x\_sq}}(r_{\sigma_t})$ and $\widetilde{\sigma\text{\_sq}}(r_{\sigma_t})$ under $\lambda_{\text{sig}} \leftarrow \mathsf{FS.challenge}$
recover $\text{var}(r_{\sigma_t})$ algebraically and discharge floor-sqrt via rps_σ
$\alpha \leftarrow \mathsf{FS.challenge}$; Y-fusion cubic multi-batched sumcheck on $[\gamma \odot \widetilde{\mathbf{x}}, \widetilde{\sigma} \odot \widetilde{\mathbf{y}}]$
push Hyrax claims to deferred accumulators $\mathsf{acc}_t, \mathsf{acc}_{td}$ (batched across LNs by PROVELNSBATCHED)

A.3 Full verifier algorithm

Algorithm 7 π-Former.$\mathsf{Verify}(\mathsf{vk}, \pi, \mathsf{inst}, \mathbf{x}_{\text{in}}, \mathbf{y})$

check $\mathsf{Com}(\mathbf{x}_{\text{in}}) = \pi.C_{\mathbf{x}_{\text{in}}}$
$\mathsf{FS.init}$; absorb $\pi.C_{\mathbf{x}_{\text{in}}}$; initialise deferred Hyrax accumulators
$C_{\text{cur}} \leftarrow \pi.C_{\mathbf{x}_{\text{in}}}$
for $\ell = 1, \ldots, L$ **do**
    mirror Phase 1 absorption (LN_1, LN_Q, LN_K, LN_{aon}, LN_2 IO + attn-norm aux)
    $C_{\mathbf{X}_{\text{mid}}} \leftarrow C_{\text{cur}} + \pi.C^\ell_{\text{LN}_{\text{aon}}.y}$; $C_{\text{cur}} \leftarrow C_{\mathbf{X}_{\text{mid}}} + \pi.C^\ell_{\mathbf{O}_{\text{ffn}}}$
end for
draw r_{td}; verify the four cross-block sumchecks $\pi_{qkv}, \pi_{op}, \pi_{ao}, \pi_{ac}$
(when normalised) verify π_{atn} at r_{norm} and π_Z
verify the global FFN Lasso and π_{ffy}, π_{ffm}
verify the bucketed range batch (one m_{com} per width); verify the model-level batched LN proof
draw r_{lm}, compute $v_{\text{lm}} = \widetilde{\mathbf{y}}(r_{\text{lm}})$, verify the LM-head sumcheck, chain the input-side claim into acc_{td}
drain deferred-accumulator batching challenges and finalise in one MSM pair per Hyrax-params group; verify all grouped batched openings
verify the global $\phi(\mathbf{Q}^{\text{n}}), \phi(\mathbf{K}^{\text{n}})$ Lasso
return ACCEPT

B Full soundness proofs

B.1 Cross-block batching

Proof of Theorem 3. The challenge η is drawn from FS after every per-block claim and every per-block witness commitment has been absorbed, so the prover is committed to $\{H_\ell\}$ before learning η. If any single claim is false, $P(\eta) = \sum_\ell \eta^{\ell-1}(\hat{H}_\ell - H_\ell)$ is a non-zero polynomial of degree $\le L-1$; by Schwartz–Zippel it vanishes for at most $L-1$ values of η, so a uniformly random η catches the inconsistency with probability $\ge 1-(L-1)/|\mathbb{F}|$. Conditional on the combined claim being honest, the inner sumcheck has soundness $n\delta/|\mathbb{F}|$, and a union bound yields the stated error. □

B.2 Homomorphic residuals

Proof of Theorem 4. Hyrax commits row-by-row by an MSM over public generators $\mathbf{g}$. By bilinearity of MSM, $C_{\mathbf{A}} + C_{\mathbf{B}} = (\text{MSM}(\mathbf{g}, A_i) + \text{MSM}(\mathbf{g}, B_i))_i = (\text{MSM}(\mathbf{g}, A_i + B_i))_i = \mathsf{Com}(\mathbf{A}+\mathbf{B})$. Binding of $\mathbf{A}+\mathbf{B}$ then follows from binding of $\mathbf{A}$ and $\mathbf{B}$ (Lemma 2). □

B.3 Range proof with zerocheck fold

Proof of Theorem 5. Each width bucket is processed independently; we argue per bucket.

Chunk-value binding. The Fiat–Shamir invariant is that m_{com} is absorbed before any per-witness sumcheck challenge $r_v^{(b)}$. Since m aggregates all chunk occurrences across all witnesses and all $C = \lceil \text{bits}/16 \rceil$ chunks per witness, the prover is committed to m before seeing any sumcheck challenge. If the prover supplied a chunk value $v^* \geq 2^{16}$, the LogUp identity [6] would require $m[v^*] \cdot T_{\text{id}}[v^*]$ to balance; since T_{id} has no entry for v^*, this contradicts the committed m (Lemma 2).

Inverse-polynomial binding. Let $\tilde{h}$ be the prover's choice of inverse-polynomial MLE, V the (committed) chunk MLE, and define $\tilde{s}(x) := \tilde{h}(x)(\alpha - V(x)) - 1$. The bucket sumcheck proves

$$\sum_x \Big[w(x)\tilde{h}(x)(1 + \beta(\alpha - V(x))) + \gamma_{\text{fold}} \cdot \text{eq}(r_{zc}, x)\tilde{s}(x) \Big] = \Lambda,$$

with Λ computed from the prover-supplied per-chunk sums and absorbed *before* γ_{fold} and r_{zc} are sampled. If $\tilde{h} \neq 1/(\alpha - V)$ on the boolean hypercube, $\tilde{s}$ is non-zero on at least one boolean point, hence non-zero as a polynomial of total degree $\leq 2n$; by Schwartz–Zippel, $\Pr_{r_{zc}}[\tilde{s}(r_{zc}) = 0] \leq 2n/|\mathbb{F}|$. Conditioning on $\tilde{s}(r_{zc}) \neq 0$, the true integrand is linear in γ_{fold} with non-zero slope, so the probability the combined sum equals Λ over a fresh γ_{fold} is at most $1/|\mathbb{F}|$. If the sum differs from Λ, the standard sumcheck rejects with probability $\geq 1 - 3n/|\mathbb{F}|$. Union-bounding yields $\leq 6n/|\mathbb{F}|$.

The pair-padding positions of h are filled with α^{-1}. On padding, $w = 0$ kills the main term, and $\alpha^{-1}\alpha - 1 = 0$ kills the zerocheck term, so padding contributes nothing to either side. Once $\tilde{h}$ is forced to the unique inverse on the hypercube, the per-chunk claims $\sum_i \tilde{h}_p[i]$ recover the true LogUp LHS, and the grand-sum check $\sum_p \text{claim}_p = \text{rhs\_claim}$ closes the argument against the multiplicity table. □

B.4 End-to-end

Proof of Theorem 8. Hybrid argument over the L blocks. *(i)* Any forged Hyrax opening yields a DL solution by Lemma 2. *(ii)* Block-level chaining is sound by Theorem 4: the sandwich-norm residual $C_{\mathbf{X}_{\text{mid}}} = C_{\mathbf{x}_{\text{in}}} + C_{\text{LN}_{\text{aon}} \cdot y}$ inherits binding. *(iii)* Per-block algebra is sound by Theorems 5, 6, 7, and 3; the committed-output Lasso is sound by Lemmas 2–1. The Lasso index-binding step ties lookup queries to the post-QK-norm tensors, which are themselves bound to the pre-norm $\mathbf{Q}, \mathbf{K}$ through the LN-batched proof for LN_Q, LN_K. *(iv)* The final layer plus LM head reduce to one LN plus one projection. A union bound over all sub-protocols yields the stated bound. For typical parameters ($L \leq 12$, $n_{\max} \leq 64$, $\delta_{\max} \leq 3$) over BN254, the statistical term is below 2^{-240} and is dominated by ε_{DL}. □